

HDOC day-17

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and with a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- .Generate the largest possible number using its digits
- .Generate the smallest possible number using its digits.
- .Swap its first and last digits.
- .Check whether it is a palindrome

Question 1

Generate the largest possible number using the digits of a positive integer.

Algorithm

Start.

- 1)Enter a positive integer.
- 2)Take each digit of the number.
- 3)Count how many times each digit occurs.
- 4)Print digits from 9 to 0.
- 5)This gives the largest possible number.
- 6)Stop.

C code

```
#include <stdio.h>
```

```
void largest(q)
{
    int n, d, a[10] = {0};

    printf("Enter number: ");
    scanf("%d", &n);

    while(n > 0)
    {
        d = n % 10;
        a[d]++;
        n = n / 10;
    }

    printf("Largest number = ");

    for(d = 9; d >= 0; d--)
        while(a[d] > 0)
        {
```

```
        printf("%d", d);  
        a[d]--;  
    }  
}
```

```
int main()  
{  
    largest();  
    return 0;  
}
```



pragya@kali: ~

Session Actions Edit View Help

GNU nano 9.0

program21.c

```
#include<stdio.h>
void largest( )
{
    int n,d,a[10]={0};
    printf("Enter number");
    scanf("%d",&n);
    while(n>0)
    {
        d=n%10;
        a[d]++;
        n=n/10;
    }
    printf("Largest number=");
    for(d=9;d>=0;d--)
    {
        while(a[d]>0)
        {
            printf("%d",d);
            a[d]--;
        }
    }
}

int main( )
{
    largest( );
    return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^N Replace

^K Cut
^U Paste

^T Execute
^J Justify

[Read 27 lines]
^C Location
^_ Go To Line

M-U Und
M-E Rec



pragya@kali: ~

Session Actions Edit View Help

```
(pragya@kali)-[~]  
$ nano program21.c
```

```
(pragya@kali)-[~]  
$ gcc program21.c -o program21
```

```
pragya@kali:~/C++$ ./program21  
Enter number: 35142  
Largest number=54321
```

```
(pragya@kali)-[~]  
$ ./program21
```

```
Enter number: 35142  
Largest number=54321
```

```
(pragya@kali)-[~]  
$
```

```
(pragya@kali)-[~]  
$ nano program20.c
```

```
(pragya@kali)-[~]  
$
```

Test Case Table			
Test Case	Input Number	Expected output	Actual output
1	3 5 1 4 2	5 4 3 2 1	5 4 3 2 1
2	1 2 3 4 5	5 4 3 2 1	5 4 3 2 1
3	9 8 7 6	9 8 7 6	9 8 7 6
4	1 1 2 3 5	5 3 2 1 1	5 3 2 1 1

Result: The program was successfully executed and generated the largest possible number using the digits of the given number.

Question 2

Generate the smallest possible number using its digits.

Algorithm:

- 1) Start.
- 2) Enter a positive integer.
- 3) Separate its digits.
- 4) Count each digit.
- 5) Print the digits from 0 to 9 in increasing order.
- 6) Display the smallest possible number.
- 7) Stop.

C code

```
#include <studio. h>
```

```
void smallest()
```

```
{
```

```
int n, d, a[10] = {0};
```

```
printf("Enter number: ");
```

```
scanf("%d", &n);
```

```
while(n > 0)
{
    d = n % 10;
    a[d]++;
    n = n / 10;
}

printf("Smallest number = ");

for(d = 0; d <= 9; d++)
    while(a[d] > 0)
    {
        printf("%d", d);
        a[d]--;
    }
}
```

Result



pragya@kali: ~

Session Actions Edit View Help

GNU nano 9.0

program22.c

```
#include<stdio.h>
void smallest()
{
    int n,d,a[10]={0};
    printf("Enter number:");
    scanf("%d",&n);
    while(n>0)
    {
        d=n%10;
        a[d]++;
        n=n/10;
    }
    printf("Smallest number=");
    for(d=0;d<=9;d++)
    {
        while(a[d]>0)
        {
            printf("%d",d);
            a[d]--;
        }
    }
}

int main( )
{
    smallest( );
    return 0;
}
```

[Read 27 lines]

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^N Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U
M-E



pragya@kali: ~

Session Actions Edit View Help

```
(pragya@kali)-[~]
```

```
$ nano program22.c
```

```
(pragya@kali)-[~] program22
```

```
$ gcc program22.c -o program22
```

```
program22.c:14:12: error: expected '}' before '}' token
```

```
program22.c:14:12: error: expected '}' before '}' token
```

```
$ ./program22
```

```
Enter number:35142
```

```
Smallest number=12345 expected '}' before '}' token
```

```
(pragya@kali)-[~]
```

```
$
```

```
(pragya@kali)-[~]
```

```
$ nano program22.c
```

```
(pragya@kali)-[~]
```

```
$
```

Test Case Table			
Test Case	Input	Expected output	Actual output
1	35142	12345	12345
2	54321	12345	12345
3	9876	6789	6789
4	53211	11235	11235

The program was successfully executed and generated the smallest possible number using the given digits.

Question 3

Swap its first and last digits.

Algorithm

Start.

- 1) Enter a positive integer.
- 2) Find the first digit.
- 3) Find the last digit.
- 4) Swap the first and last digits.
- 5) Display the new number.
- 6) Stop.

C code

```
void swap()
```

```
{
```

```
    int n, first, last, p=1, temp;
```

```
    printf("Enter number: ");
```

```
    scanf("%d",&n);
```

```
last = n % 10;
temp = n;

while(temp >= 10)
{
    temp = temp / 10;
    p = p * 10;
}

first = temp;

n = n - first*p - last + last*p + first;

printf("After swap = %d",n);
}
```

Result



pragya@kali: ~

Session Actions Edit View Help

GNU nano 9.0

program23.c

```
#include<stdio.h>
void largest( )
{
    int n,d,a[10]={0};
    printf("Enter number:");
    scanf("%d",&n);
    while(n>0)
    {
        d=n%10;
        a[d]++;
        n=n/10;
    }
    printf("Largest number=");
    for(d=9;d>=0;d--)
    while(a[d]>0)
    {
        printf("%d",d);
        a[d]--;
    }
}
int main( )
{
    largest( );
    return 0;
}
```

[Read 25 lines]

^G Help
^X Exit

^O Write Out
^R Read File

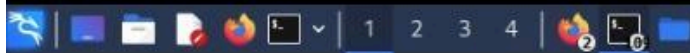
^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo



pragya@kali: ~

Session Actions Edit View Help

```
(pragya@kali)-[~]  
$ nano program23.c
```

```
(pragya@kali)-[~] program23  
$ gcc program23.c -o program23
```

```
(pragya@kali)-[~]  
$ ./program23
```

Enter number:12345
Largest number=54321

```
(pragya@kali)-[~]  
$
```

classmate
 Date _____
 Page _____

Test Case Table

Test Case	Input	Expected output	Actual output
1	12345	52341	52341
2	4567	7564	7564
3	9876	6879	6879

Test Case Table

The program was successfully executed and swapped the first and last digits of the given number.

Question 4

Check whether it is a palindrome.

Algorithm

- 1) Start.
- 2) Enter a positive integer.
- 3) Store the number in a temporary variable.
- 4) Reverse the number.
- 5) Compare the original number with the reversed number.
- 6) If both are equal, print Palindrome.
- 7) Otherwise, print Not Palindrome.
- 8) Stop.

C code

```
void palindrome()
```

```
{
```

```
    int n, temp, rev=0, d;
```

```
    printf("Enter number: ");
```

```
    scanf("%d",&n);
```

```
temp=n;

while(temp>0)
{
    d=temp%10;
    rev=rev*10+d;
    temp=temp/10;
}

if(n==rev)
    printf("Palindrome");
else
    printf("Not Palindrome");
}
```



```
pragya@kali: ~  
Session Actions Edit View Help  
GNU nano 9.0 program24.c  
#include<stdio.h>  
void palindrome( )  
{  
    int n,temp,rev=0,d;  
    printf("Enter number");  
    scanf("%d",&n);  
    temp=n;  
    while(temp>0)  
    {  
        d=temp%10;  
        rev=rev*10+d;  
        temp=temp/10;  
    }  
    if(n==rev)  
        printf("Palindrome");  
    else  
        printf("not palindrome");  
}  
int main( )  
{  
    palindrome( );  
    return 0;  
}  
[ Read 24 lines ]  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```



pragya@kali: ~

Session Actions Edit View Help

```
(pragya@kali)-[~]  
$ nano program24.c
```

```
(pragya@kali)-[~] program19  
$ gcc program24.c -o program24
```

```
(pragya@kali)-[~]  
$ ./program24
```

Enter number121
Palindrome

```
(pragya@kali)-[~]  
$
```

Test Case Table			
Test Case	Input	Expected output	Actual Output
1	121	Palindrome	Palindrome
2	12321	Palindrome	Palindrome
3	123	Not Palindrome	Not Palindrome

Result

The program was successfully executed and checked whether the given number is palindrome or not.

Thank you

Pragya pandey

Batch 15

Sap id 590034352